

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q. No	Question	Bloom's Level
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze

5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze
8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

Code of Conduct:

I S.JAYA SHREE(192524146) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.JAYA SHREE

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Sequential file organization stores records physically ordered by a key field, whereas heap (unordered/pile) file organization stores records in no particular order, typically in the sequence they were inserted.

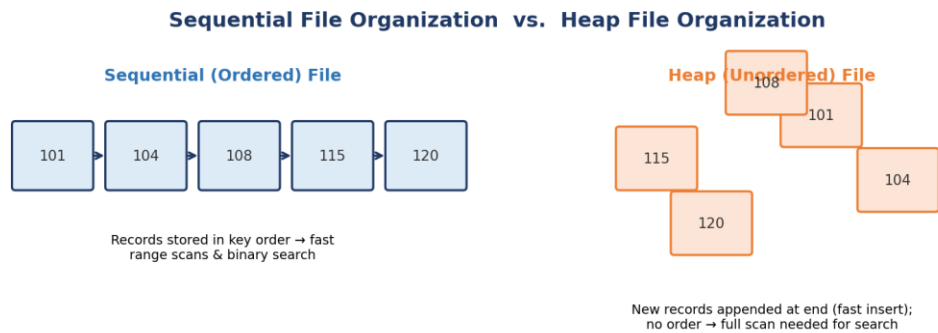


Fig. 1 — Sequential vs. Heap file layout

Aspect	Sequential File Organization	Heap File Organization
Data Insertion	Costly — the correct sorted position must be located and subsequent records shifted, or an overflow area is used and periodically merged.	Very fast — new record is simply appended at the end of the file ($O(1)$).
Data Deletion	Requires shifting records to close the gap, or marking a tombstone and reorganizing later.	Fast — record is marked deleted (tombstone) or space reused; no reordering needed.
Retrieval (Search)	Efficient — supports binary search, $O(\log n)$, and fast range queries since order is preserved.	Inefficient — requires a linear scan, $O(n)$, since there is no ordering to exploit.
Best Use Case	Batch processing, reporting, range queries (e.g., payroll runs).	Systems with heavy, frequent insertions and few searches (e.g., transaction logs).

Example:

A payroll system that processes employees in Employee_ID order benefits from sequential organization. A temporary transaction table receiving a large number of new records benefits from heap organization.

Conclusion:

Sequential organization is better when ordered and range-based retrieval is important, whereas heap organization is preferable when fast insertion is the main requirement.

Q2. Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

A clustered index determines the physical storage order of table rows, while a non-clustered index maintains a separate structure of pointers to the actual data rows, leaving the physical order of the table unaffected.

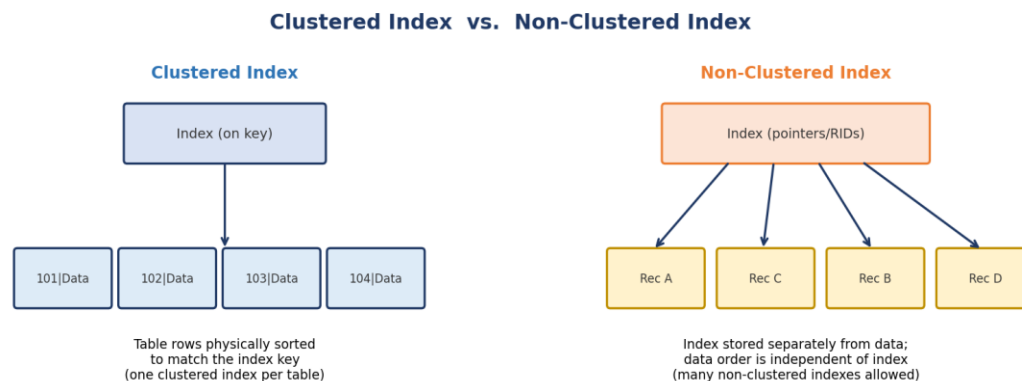


Fig. 2 — Clustered vs. Non-clustered index structure

Aspect	Clustered Index	Non-Clustered Index
Physical Order	Table data is physically sorted and stored according to the index key.	Table data order is independent of the index; index is a separate structure with row pointers/RIDs.
Number per Table	Only one per table (since data can be sorted only one way).	Multiple non-clustered indexes can exist on the same table.

Aspect	Clustered Index	Non-Clustered Index
Storage Overhead	Lower — no separate data copy; index and data are the same structure.	Higher — stores key values plus pointers to actual rows, in addition to the base table.
Retrieval Speed	Very fast for range queries and queries on the clustering key (data is already ordered).	Fast for point lookups; range queries may need extra I/O to fetch actual rows via pointers.
Example	A `student_id` primary key in a `Students` table where rows are stored in `student_id` order.	An index on `email` or `department` column in the same `Students` table for quick lookup without reordering rows.

Database Example:

Consider an Employee table containing:

Employee(Emp_ID, Name, Department, Salary)

If the table is organized using Emp_ID, records can be stored according to the clustered key. A separate index on Department can act as a non-clustered or secondary index to quickly locate employees belonging to a particular department.

For example:

```
CREATE INDEX idx_department  
ON Employee(Department);
```

This index allows the DBMS to search employees based on Department without scanning every row.

Conclusion:

Clustered indexing is especially useful for range and ordered retrieval, while non-clustered indexing provides flexibility by allowing multiple search paths over the same table.

Q3. Compare primary indexing and secondary indexing based on storage requirements and search performance.

Primary indexing is built on the ordering (primary) key of a sequentially organized file, typically using one index entry per data block. Secondary indexing is built on a non-ordering field and typically requires one index entry per record.

Aspect	Primary Index	Secondary Index
Basis	Created on the primary key of an ordered (sequential) data file.	Created on a non-primary, non-ordering candidate key or a frequently searched field.
Index Entries	Sparse — one entry per data block, since the file is already sorted on the key.	Dense — usually one entry per record, since the underlying file is not sorted on this field.
Storage Requirement	Lower, because fewer index entries are needed (block-level granularity).	Higher, because an entry must exist for every record to locate it uniquely.
Search Performance	Very fast — combines binary search on the sparse index with a short scan within the target block.	Slightly slower and more storage-intensive, but still far faster than a full linear scan; essential when the search field is not the ordering key.

Example:

Consider:

Student(Student_ID, Name, Department, Email)

A primary index can be based on Student_ID. A secondary index may be created on Department to retrieve students belonging to a particular department.

```
CREATE INDEX idx_student_department
ON Student(Department);
```

Analysis:

Primary indexing consumes less additional storage because the data itself is organized using the primary key. Secondary indexing consumes more space because an additional index structure must maintain references to records.

Conclusion:

Primary indexes provide excellent performance for primary-key retrieval with lower storage overhead, while secondary indexes use additional storage but provide more flexible search options.

Q4. Analyze the differences between static hashing and dynamic hashing in database systems.

Static hashing uses a fixed number of buckets determined at file-creation time, mapping keys through a hash function whose range does not change. Dynamic hashing (e.g., extendible or linear hashing) allows the number of buckets to grow or shrink automatically as the data volume changes.

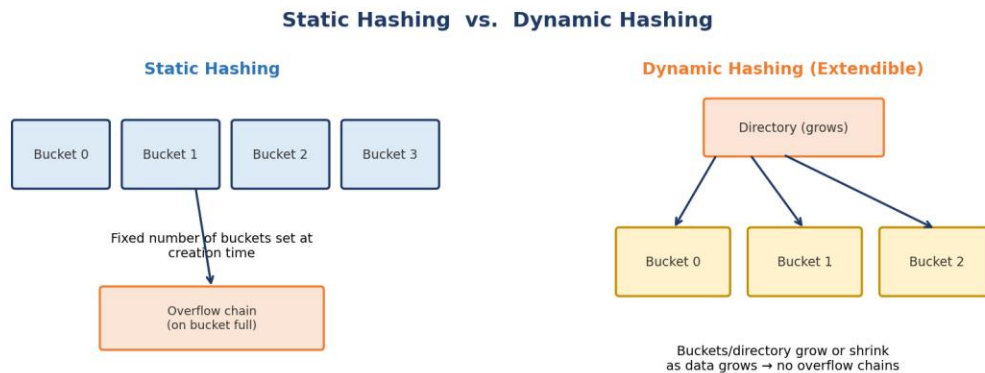


Fig. 3 — Static vs. Dynamic hashing

Aspect	Static Hashing	Dynamic Hashing
Bucket Count	Fixed at design time; does not change as data grows or shrinks.	Adjusts dynamically at run time in response to the actual data volume.
Overflow Handling	Bucket overflow is handled with overflow chains, which degrade search performance over time.	Buckets split/merge as needed, largely avoiding long overflow chains.
Performance Over Time	Degrades as the file grows beyond its originally planned capacity.	Remains consistently efficient because the structure adapts to the current data size.
Reorganization Cost	Requires a full, expensive rehash of the entire file if resized.	Incremental — only affected buckets/directory entries are restructured.
Best Suited For	Databases with a known, stable, and predictable size.	Databases with unpredictable or rapidly changing data volumes.

Example:

Suppose a student initially contains 1,000 records. In static hashing, a fixed number of buckets is allocated. If the database grows to 50,000 records, many bucket overflows may occur.

With dynamic hashing, additional buckets can be created as the number of records increases, reducing overflow and maintaining faster retrieval.

Analysis:

Static hashing is suitable for databases whose size does not change significantly.

Dynamic hashing is more appropriate for modern applications where records are frequently inserted and deleted.

Conclusion:

Static hashing offers simplicity and low overhead, whereas dynamic hashing offers superior scalability and more consistent performance for growing databases.

Q5. Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

A B-Tree stores both keys and associated data (or record pointers) in internal as well as leaf nodes. A B+ Tree restricts actual data references to the leaf nodes only, using internal nodes purely as a routing/index structure, with leaf nodes additionally linked in a sequential chain.

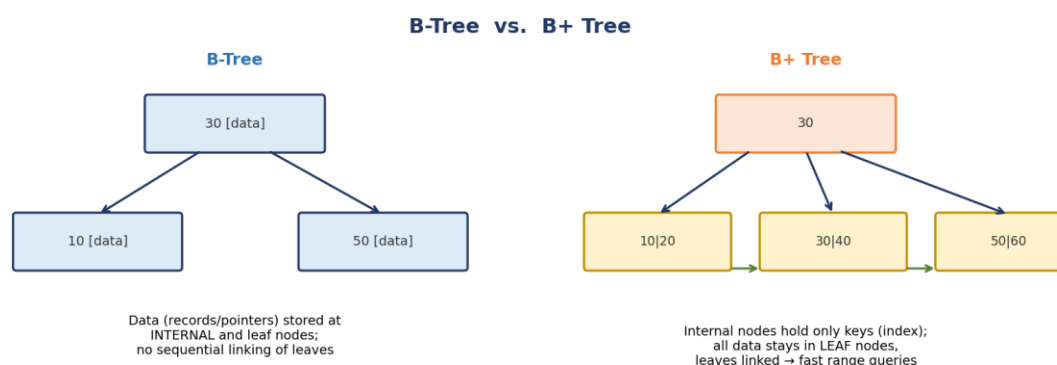


Fig. 4 — B-Tree vs. B+ Tree structure

Aspect	B-Tree	B+ Tree
Data Storage Location	Data/record pointers stored in both internal and leaf nodes.	Data/record pointers stored only in leaf nodes; internal nodes hold keys only.

Aspect	B-Tree	B+ Tree
Leaf-Node Linking	Leaf nodes are not linked sequentially.	Leaf nodes are linked in a doubly/singly linked list, enabling fast ordered traversal.
Range Query Performance	Slower — the tree must be re-traversed from the root for each range boundary.	Much faster — once the first key is found, remaining results are read by following leaf links.
Fan-out / Tree Height	Lower fan-out per node (since nodes also carry data), leading to a slightly taller tree.	Higher fan-out (internal nodes carry only keys), giving a shorter, flatter tree for the same data volume.
Typical Usage	General-purpose indexing where range scans are infrequent.	Preferred by virtually all modern RDBMS engines (e.g., relational table indexes) due to superior range-query performance.

Example:

Consider millions of banking transaction records arranged by `Transaction_ID`. A B+ Tree allows the DBMS to locate the starting record quickly and then traverse linked leaf nodes for a range such as `Transaction_ID` 10000 to 20000.

Analysis:

For large databases, minimizing disk access is important. B+ Trees have high fan-out, allowing more keys per internal node and reducing the number of disk pages that may need to be accessed.

Conclusion:

Both structures provide efficient searching, but B+ Tree indexing is generally more suitable for large-scale database systems, especially where range queries and sequential access are common.

Q6. Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Both are dynamic hashing schemes, but they manage bucket growth differently. Linear hashing splits buckets in a predetermined, round-robin order without needing a directory, while extendible hashing maintains a directory of pointers to buckets and doubles that directory when needed.

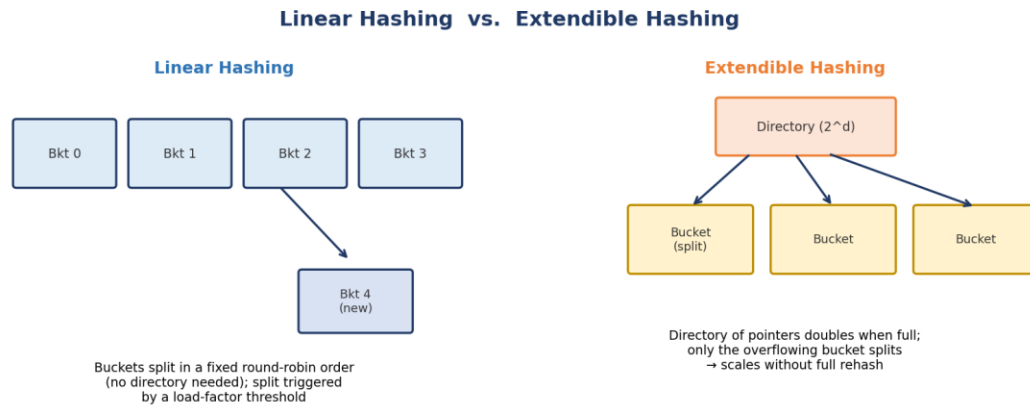


Fig. 5 — Linear vs. Extendible hashing

Aspect	Linear Hashing	Extendible Hashing
Directory Requirement	No directory needed — buckets are split in a fixed, predictable sequence.	Requires a directory of 2^d pointers to buckets, which must be maintained and searched.
Split Trigger	A split occurs when a load-factor threshold is crossed, regardless of which bucket actually overflowed (may allow temporary overflow chains).	A split occurs precisely on the bucket that overflows, using local depth vs. global depth comparison.
Directory Growth	Not applicable (no directory).	Directory doubles in size when the global depth needs to increase, which can be a relatively costly step.
Space Utilization	Generally simpler and avoids directory overhead, but may tolerate brief overflow chains.	Very predictable single-bucket-access performance, at the cost of directory memory overhead.

Aspect	Linear Hashing	Extendible Hashing
Scalability	Scales smoothly with simple bucket-by-bucket growth; well suited to steady, incremental growth.	Scales well for rapidly and unpredictably growing data, since only the directory needs to expand while data movement is localized.

Example:

In linear hashing, when records increase, buckets are split one after another gradually instead of immediately doubling the entire structure.

In extendible hashing, directory entries point to buckets. When a bucket becomes full, the bucket may split, and the directory may double if required.

Analysis:

Linear hashing minimizes directory overhead and provides smooth growth. Extendible hashing offers more flexible bucket selection and can provide very efficient lookup but requires additional memory for the directory.

Conclusion:

Linear hashing is preferred when lower memory overhead is important, while extendible hashing is suitable for rapidly growing systems requiring efficient dynamic bucket management.

Q7. Compare dense indexing and sparse indexing with respect to memory usage and access speed.

A dense index contains an index entry for every record in the data file, whereas a sparse index contains an entry only for some records — typically one per disk block — relying on the fact that the data file is sorted.

Aspect	Dense Index	Sparse Index
Index Entries	One entry for every single record in the data file.	One entry per block (or group of records); many records share a single index entry.
Memory / Storage Usage	Higher — index size grows in direct proportion to the number of records.	Lower — index size grows in proportion to the number of blocks, which is much smaller than the number of records.

Aspect	Dense Index	Sparse Index
Access Speed (Point Lookup)	Very fast — the index directly locates the exact record without any further scanning.	Slightly slower — the index locates the block, and then a short local scan within that block is needed.
Applicability	Can be built on both ordered and unordered files.	Can only be built on a file that is already sorted on the indexed field.

Example:

Suppose a database has 10,000 student records.

A dense index may contain index entries corresponding to all 10,000 records or search-key occurrences.

A sparse index might store only one entry for each disk block. If each block contains 100 records, approximately 100 index entries may be enough.

Analysis:

Dense indexing trades increased storage for faster retrieval. Sparse indexing reduces memory requirements but requires an additional search after the correct block is identified.

Conclusion:

Dense indexing provides higher access speed, while sparse indexing provides better memory efficiency. The choice depends on whether retrieval speed or storage efficiency is more important.

Q8. Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Indexed Sequential Access Method (ISAM) combines sequential storage of records with a multi-level index for faster access. Direct (hashed) file organization computes a record's storage address directly from its key using a hash function, without maintaining sequential order.

Aspect	Indexed Sequential (ISAM)	Direct (Hashed) File Organization
Advantages	Supports both fast direct access (via the index) and	Extremely fast average-case direct access (near $O(1)$) since

Aspect	Indexed Sequential (ISAM)	Direct (Hashed) File Organization
	efficient sequential/range access (records remain sorted); good for reporting plus lookups.	the address is computed, not searched, for exact-match queries.
Limitations	Insertions can cause overflow areas to grow, degrading performance until the file is reorganized; index itself consumes extra storage.	Provides no support for range queries or sorted access, since there is no key ordering; performance can degrade with collisions.
Insert/Delete Cost	Moderate — may require overflow-area management or periodic reorganization.	Low on average, but collision resolution (chaining/open addressing) adds overhead as the file fills up.
Best Suited For	Applications needing both direct lookups and sequential/range processing (e.g., banking master files).	Applications needing very fast exact-match key lookups with little or no need for range queries (e.g., session/key-value lookups).

Example:

A bank may use indexed sequential organization for customer records because records may need to be processed both individually and sequentially.

A direct organization approach can be useful where an account number is transformed into a storage location using a hash function.

Advantages of Indexed Sequential Organization:

It supports both sequential access and direct indexed access, performs well for range queries, and allows ordered processing.

Limitations:

It requires additional index storage, overflow areas may develop after repeated insertions, and periodic reorganization may be necessary.

Advantages of Direct Organization:

Exact-match retrieval is extremely fast and insertion can be efficient.

Limitations:

It performs poorly for range queries and may suffer from hash collisions and bucket overflow.

Conclusion:

Indexed sequential organization is more versatile, while direct organization is preferable when exact-match retrieval dominates the workload.

Q9. Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

A single-level index maps index entries directly to data blocks. A multi-level index builds an index over the index itself (and can recurse further), forming a hierarchy so that the outermost level is small enough to be held in main memory.

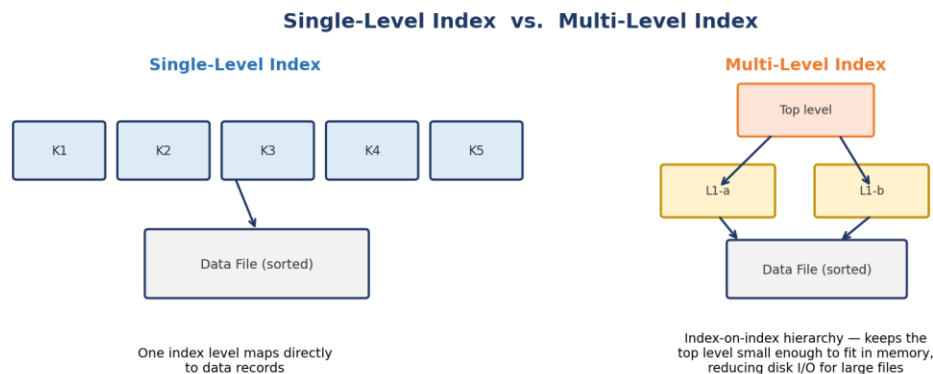


Fig. 6 — Single-level vs. Multi-level index hierarchy

Aspect	Single-Level Index	Multi-Level Index
Structure	One flat index directly referencing data blocks.	A hierarchy of indexes, where each level indexes the level below it, down to the data file.
Search Cost (large files)	Binary search on a large single-level index still requires $\log_2(n)$ block accesses, which grows as the file grows.	Reduces the number of disk accesses because the top level is small enough to fit in memory; only a few levels need to be traversed.
Scalability	Becomes inefficient once the index itself grows too large to fit comfortably in memory.	Scales well to very large files, since additional levels can be added as the data grows (similar in spirit to B+ Tree structures).

Aspect	Single-Level Index	Multi-Level Index
Maintenance Overhead	Simpler to maintain — only one structure to update on insert/delete.	More complex to maintain — updates may need to propagate through multiple index levels.

Example:

Suppose a database contains 1,000,000 employee records. A single-level index itself may become very large and require several disk accesses.

With multi-level indexing, a smaller upper-level index points to blocks of the lower-level index. Therefore, only a small number of index blocks need to be accessed before reaching the desired record.

Analysis:

Multi-level indexing applies the same idea recursively until the highest-level index becomes small enough to search efficiently.

B-Tree and B+ Tree structures implement this concept efficiently by maintaining balanced hierarchical index levels.

Conclusion:

Single-level indexing is simpler and suitable for smaller datasets. Multi-level indexing offers better scalability and efficient retrieval for large-scale databases.

Q10. Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Hashing techniques compute a record's location directly from its key value, while indexing techniques (tree-based structures such as B+ Trees, or sequential indexes) maintain an ordered or hierarchical lookup structure. Their relative performance differs sharply depending on the type of query being evaluated.

Aspect	Exact-Match Queries	Range Queries
Hashing Techniques	Excellent — average-case $O(1)$ lookup, since the hash function computes the exact bucket address directly.	Poor — hashing destroys key ordering, so retrieving a range typically requires scanning the entire structure.

Aspect	Exact-Match Queries	Range Queries
Indexing Techniques (e.g., B+ Tree)	Good — $O(\log n)$ lookup via tree traversal; slightly slower than hashing but still efficient.	Excellent — sorted/linked structure (e.g., B+ Tree leaf chain) allows sequential retrieval of all qualifying records after one initial search.
Storage Overhead	Generally low to moderate; overhead grows with collision-handling structures.	Moderate to high, due to internal index nodes maintained purely for navigation.
Update Cost	Low, unless rehashing/bucket-splitting is triggered.	Moderate — insinuates rebalancing or node splits (e.g., in B+ Trees) to preserve order.

Exact-Match Query Consider:

```
SELECT *
```

```
FROM Employee
```

```
WHERE Employee_ID = 105;
```

For an exact-match query, hashing can be highly efficient because the hash function calculates the bucket in which the required record should be located.

An index on Employee_ID also performs efficiently, but a tree index generally requires traversal through multiple nodes.

Therefore, for pure exact-match queries:

Hashing → generally excellent

Range Query Consider:

```
SELECT *
```

```
FROM Employee
```

```
WHERE Salary BETWEEN 50000 AND 80000;
```

Hashing is inefficient for this query because hash values do not preserve the natural ordering of salaries.

An ordered index such as a B+ Tree can locate the first qualifying salary and then continue through adjacent leaf entries.

Therefore, for range queries:

B+ Tree / ordered indexing → generally superior

Performance Evaluation

Hashing is ideal for applications where most searches use equality conditions, such as:

WHERE Account_No = 12345

Indexing is more versatile because it supports exact searches, range searches, ordering and sequential retrieval.

For example:

**CREATE INDEX idx_salary
ON Employee(Salary);**

This index can improve searches involving salary values, particularly range-based retrieval.

Conclusion

No single file organization, indexing, or hashing technique is universally optimal; each involves a trade-off between insertion/deletion cost, storage overhead, and retrieval speed for a specific query pattern. Ordered structures (sequential files, B+ Trees, multi-level indexes) excel at range queries and sorted access, while unordered/direct structures (heap files, hashing) excel at fast insertion and exact-match lookups. Effective database design under CO4 therefore requires selecting — or combining — these techniques based on the expected balance of read, write, exact-match, and range-query operations in the target application.